



MoveHard Sports

Dokumentacja flow aplikacji i widoków

NEXT.JS 16 APP ROUTER

SUPABASE AUTH

TYPESCRIPT

TAILWIND CSS

ZOD

Data: 15 maja 2026 | Autor: Dominik Kwaczynski

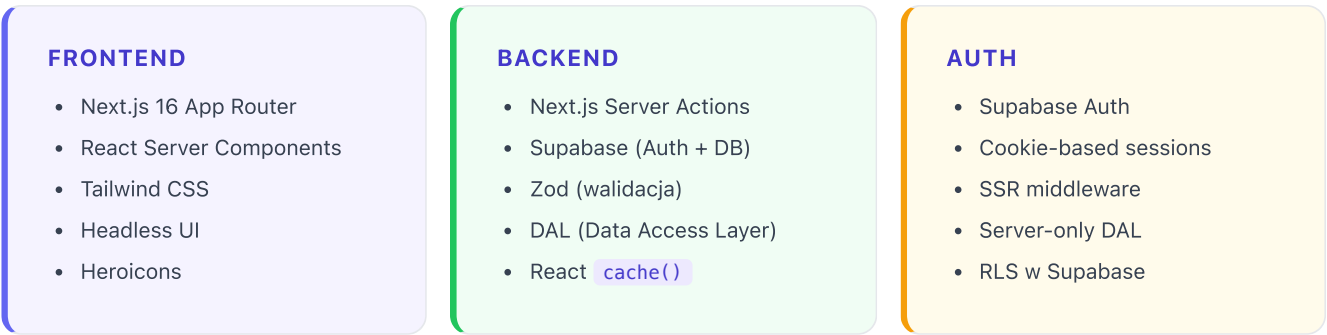
Spis treści

#	SEKCJA
1	Przegląd systemu i architektura
2	Modele danych
3	Mapa routingu
4	Flow: Nowy użytkownik (rejestracja → onboarding)
5	Flow: Logowanie
6	Flow: Dashboard — Brand
7	Flow: Kampanie — CRUD
8	Guardy i przekierowania
9	Server Actions
10	Opisy widoków

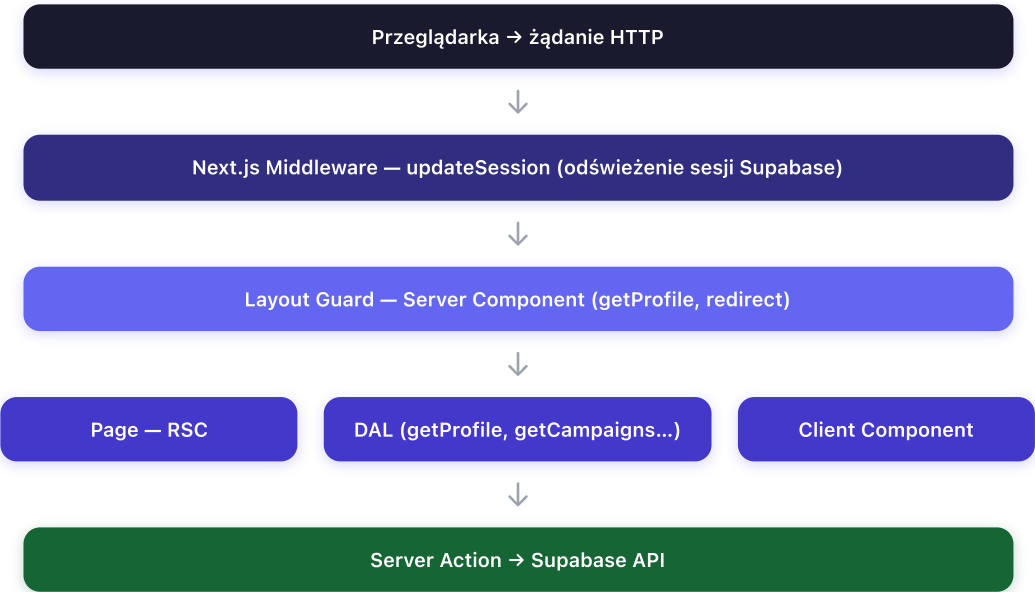
1. Przegląd systemu i architektura

MHS (MoveHard Sports) to platforma łącząca marki sportowe z atletami. Marki publikują kampanie, atleci (w przyszłości) mogą aplikować. Aplikacja zbudowana jest w Next.js 16 z App Routerem po stronie serwera, autoryzacja i baza danych oparte są na Supabase.

Stack technologiczny



Warstwy aplikacji



Typy kont

ATLETA ATHLETE

- Imię i nazwisko (first_name, last_name)
- Dyscypliny sportowe
- W przyszłości: aplikowanie na kampanie

MARKA BRAND

- Nazwa marki (brand_name)
- Dyscypliny zainteresowań
- CRUD kampanii marketingowych

2. Modele danych

Profile (discriminated union)

Profil użytkownika jest przechowywany w tabeli `profiles` w Supabase. TypeScript używa discriminated union na polu `account_type`.

```
type BaseProfile = {
  id: string
  account_type: 'athlete' | 'brand'
  onboarding_completed: boolean
  created_at: string
  updated_at: string
  disciplines: Discipline[] | null
}

type AthleteProfile = BaseProfile & {
  account_type: 'athlete'
  first_name: string
  last_name: string
  brand_name: null
}

type BrandProfile = BaseProfile & {
  account_type: 'brand'
  brand_name: string
  first_name: null
  last_name: null
}

type Profile = AthleteProfile | BrandProfile
```

Campaign

```
type CampaignStatus = 'draft' | 'active' | 'closed' | 'archived'

type Campaign = {
  id: string
  brand_id: string // FK → profiles.id
  title: string // max 200 znaków
  description: string // max 5000 znaków
  status: CampaignStatus
  disciplines: Discipline[] | null
  created_at: string
  updated_at: string
}
```

Discipline

```
const DISCIPLINE_IDS = ['crossfit', 'triathlon', 'motorsports'] as const

type Discipline = 'crossfit' | 'triathlon' | 'motorsports'
```

Walidacja (Zod schemas)

SCHEMA	POLE	REGUŁY
SignUp	email	email valid
	password	min 8 znaków, litera + cyfra
	accountType	'athlete' 'brand'
SignIn	email	email valid
	password	min 1 znak
OnboardingAthlete	first_name	wymagane, max 100
	last_name	wymagane, max 100
	disciplines	min 1 dyscyplina
OnboardingBrand	brand_name	wymagane, max 100
	disciplines	min 1 dyscyplina
Campaign	title	min 3, max 200 znaków
	description	min 10, max 5000 znaków
	disciplines	min 1 dyscyplina

3. Mapa routingu

Struktura katalogów (App Router)

```
src/app/
├─ layout.tsx                // Root layout (html, body, fonty)
├─ (main)/                  // Publiczne strony + nagłówek
│   └─ layout.tsx           // Header z nawigacją publiczną
│   └─ page.tsx             // / (homepage)
│   └─ (customer)/
│       └─ login/page.tsx    // /login
│       └─ register/page.tsx // /register
│       └─ onboarding/
│           └─ page.tsx       // /onboarding
│           └─ onboarding-form.tsx
└─ dashboard/               // Chroniona strefa (auth required)
    └─ layout.tsx           // Guard: login + onboarding
    └─ page.tsx             // /dashboard
    └─ (brand)/             // Guard: tylko account_type='brand'
        └─ layout.tsx
        └─ campaigns/
            └─ page.tsx       // /dashboard/campaigns
            └─ new/page.tsx   // /dashboard/campaigns/new
            └─ [id]/
                └─ page.tsx    // /dashboard/campaigns/:id
                └─ edit/page.tsx // /dashboard/campaigns/:id/edit
```

Tabela routów

ŚCIEŻKA URL	KOMPONENT	GUARD	DOSTĘP
/	(main)/page.tsx	brak	PUBLICZNY
/login	(customer)/login/page.tsx	brak	PUBLICZNY
/register	(customer)/register/page.tsx	brak	PUBLICZNY
/onboarding	(customer)/onboarding/page.tsx	profile ≠ null; onboarding_completed=false	AUTH REQUIRED
/dashboard	dashboard/page.tsx	profile; onboarding_completed=true	AUTH REQUIRED
/dashboard/campaigns	(brand)/campaigns/page.tsx	profile; brand only	BRAND ONLY

<code>/dashboard/campaigns/new</code>	<code>(brand)/campaigns/new/page.tsx</code>	profile; brand only	BRAND ONLY
<code>/dashboard/campaigns/:id</code>	<code>(brand)/campaigns/[id]/page.tsx</code>	profile; brand only; campaign.brand_id = user.id	BRAND ONLY
<code>/dashboard/campaigns/:id/edit</code>	<code>(brand)/campaigns/[id]/edit/page.tsx</code>	profile; brand only; campaign.brand_id = user.id	BRAND ONLY

4. Flow: Nowy użytkownik (rejestracja → onboarding)

Diagram flow



Szczegóły kroku: /register

- 1 Użytkownik wybiera typ konta przez przyciski **Athlete** / **Brand** — wartość trafia do ukrytego pola `account-type`
- 2 Wpisuje email i hasło (min 8 znaków, litera + cyfra)
- 3 Submit wywołuje Server Action `signUp`. Zod waliduje dane — błędy wracają jako `state.errors` i wyświetlają się przy polach
- 4 Sukces: `supabase.auth.signUp()` z metadana `account_type` w `user_metadata`
- 5 Automatyczny redirect na `/onboarding`

Szczegóły kroku: /onboarding

- 1 Server Component pobiera profil przez DAL. Jeśli brak profilu — redirect do `/login`. Jeśli onboarding już ukończony — redirect do `/dashboard`
- 2 Formularz renderuje się warunkowo wg `account_type`
- 3 Komponent **DisciplinePicker** — toggle buttons, przechowuje stan w `useState(Set)`, emituje ukryte pola `input[name=disciplines]`
- 4 Server Action `completeOnboarding` waliduje i wywołuje `profiles.update()` z polami odpowiednimi dla typu konta
- 5 Redirect na `/dashboard`

5. Flow: Logowanie



Obsługa błędów

BŁĘDY WALIDACJI ZOD

- Nieprawidłowy format email
- Puste hasło

Wyświetlane przy konkretnych polach formularza

BŁĄD SUPABASE

- Nieprawidłowe dane logowania
- Konto nie istnieje

Ogólny komunikat: „Invalid email or password.”

Wylogowanie

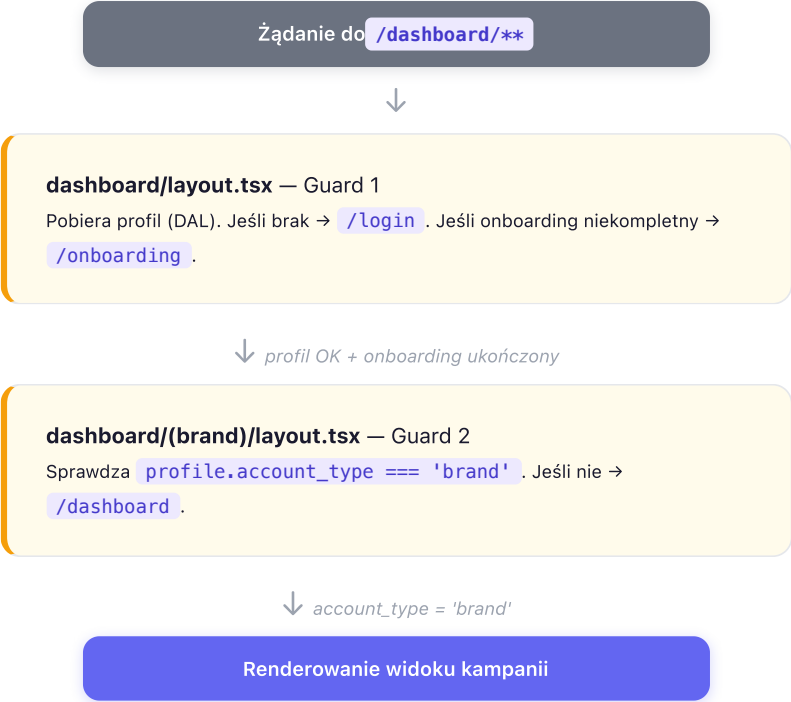
- 1 Przycisk „SIGN OUT” w headerze dashboardu (widoczny tylko gdy `showSignOut=true`)
- 2 Form POST wywołuje Server Action `signOut`
- 3 `supabase.auth.signOut()` → czyszczenie cookies
- 4 Redirect na `/` (homepage)

Zachowanie nagłówka (Header)

STAN SESJI	PRAWA STRONA NAGŁÓWKA
Niezałogowany (strona publiczna)	Przyciski: <code>SIGN IN</code> , <code>JOIN</code>
Zalogowany (strona publiczna)	Link: <code>DASHBOARD</code>

6.Flow: Dashboard — Brand

Warstwy ochrony (guards)



Nawigacja w dashboardzie

Header w layoucie dashboard renderuje nawigację z linkami:

My Campaigns → /dashboard/campaigns

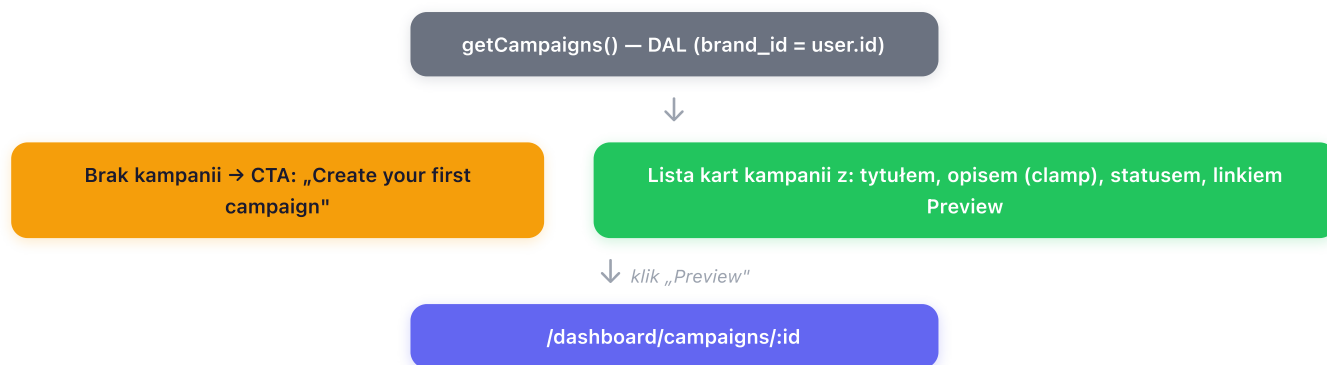
Lista kampanii marki

My Profile → #

Planowane — jeszcze niezaimplementowane

7. Flow: Kampanie — CRUD

Lista kampanii `/dashboard/campaigns`



Tworzenie kampanii `/dashboard/campaigns/new`

- 1 Klik „New Campaign” na liście. Strona renderuje **CampaignForm** bez danych wstępnych
- 2 Pola: **Title** (max 200), **Description** (textarea, max 5000), **DisciplinePicker** (min 1)
- 3 Server Action `createCampaign` : walidacja Zod → sprawdzenie profilu brand → `campaigns.insert()`
- 4 Sukces: redirect na `/dashboard/campaigns`

Szczegóły kampanii `/dashboard/campaigns/:id`

Server Component. Pobiera kampanię przez `getCampaign(id)` (filtr: `brand_id = user.id`). Jeśli nie znaleziono — `notFound()`.

Wyświetla:

- Tytuł + status (badge)
- Opis (whitespace-pre-wrap)
- Dyscypliny jako pill-badges z labelami z `DISCIPLINES`
- Daty: Created, Last updated
- Przyciski akcji: **Edit** (link) + **Delete** (form + confirm dialog)

Edycja kampanii `/dashboard/campaigns/:id/edit`

- 1 Server Component pobiera kampanię i przekazuje ją jako prop do **CampaignForm**
- 2 Formularz prefilluje pola przez `defaultValue={campaign?.title}` itd.

- 3 Server Action `updateCampaign(id)` (partial application przez `.bind(null, id)`): walidacja → `campaigns.update()` z filtrem `eq('brand_id', profile.id)`
- 4 Sukces: redirect na `/dashboard/campaigns`

Usuwanie kampanii

- 1 DeleteButton renderuje `<form>` z ukrytym polem `id`
- 2 Click handler pokazuje `window.confirm()` — jeśli użytkownik anuluje, `e.preventDefault()`
- 3 Server Action `deleteCampaign`: weryfikacja profilu brand → `campaigns.delete()` z filtrem `brand_id`
- 4 Redirect na `/dashboard/campaigns`

Bezpieczeństwo danych kampanii (RLS)

Wszystkie zapytania do tabeli `campaigns` zawierają filtr `.eq('brand_id', user.id)` — użytkownik może odczytać i zmodyfikować wyłącznie **swoje kampanie**. RLS w Supabase stanowi drugi poziom ochrony.

8. Guardy i przekierowania

Tabela guardów

MIEJSCE	WARUNEK	AKCJA
onboarding/page.tsx	!profile	redirect /login
onboarding/page.tsx	profile.onboarding_completed	redirect /dashboard
onboarding/page.tsx	account_type ≠ 'athlete' && ≠ 'brand'	redirect /login
dashboard/layout.tsx	!profile	redirect /login
dashboard/layout.tsx	!profile.onboarding_completed	redirect /onboarding
dashboard/(brand)/layout.tsx	!profile profile.account_type ≠ 'brand'	redirect /dashboard
campaigns/[id]/page.tsx	!campaign (lub nie należy do usera)	notFound() → 404
campaigns/[id]/edit/page.tsx	!campaign	notFound() → 404
Server Action: createCampaign	!profile account_type ≠ 'brand'	return { errors: { general: 'Unauthorized.' } }
Server Action: updateCampaign	!profile account_type ≠ 'brand'	return { errors: { general: 'Unauthorized.' } }
Server Action: deleteCampaign	!profile account_type ≠ 'brand'	return { error: 'Unauthorized.' }

Middleware Supabase

Plik: src/lib/supabase/middleware.ts

Wywoływany przy każdym żądaniu. Odświeża sesję Supabase (tokeny JWT w cookies) bez blokowania ruchu — guard logiki routingu znajduje się w layoutach, nie w middleware.

DAL — Data Access Layer

src/lib/dal.ts jest oznaczony jako server-only. Eksportuje funkcje opakowane w React.cache() — zapytania są deduplikowane w ramach jednego renderowania:

FUNKCJA	OPIS
getUser()	Pobiera bieżącego użytkownika z Supabase Auth
getProfile()	Pobiera profil z tabeli profiles
getCampaigns()	Lista kampanii marki (filtr: brand_id = user.id, sortowanie: created_at DESC)

`getCampaign(id)`

Pojedyncza kampania (filtr: `id + brand_id`)

9. Server Actions

auth.ts — akcje autoryzacji

AKCJA	WEJŚCIE	SUKCES	BŁĄD
signUp	accountType, email, password	redirect /onboarding	state.errors (Zod + Supabase)
signIn	email, password	redirect /dashboard lub /onboarding	state.errors.general
signOut	—	redirect /	brak (ignorowany)
completeOnboarding	first_name/last_name lub brand_name + disciplines	redirect /dashboard	state.errors (Zod + Supabase)

campaigns.ts — akcje kampanii

AKCJA	WEJŚCIE	SUKCES	BŁĄD
createCampaign	title, description, disciplines[]	redirect /dashboard/campaigns	state.errors
updateCampaign(id)	title, description, disciplines[]	redirect /dashboard/campaigns	state.errors
deleteCampaign	id (hidden field)	redirect /dashboard/campaigns	state.error (string)

Wzorzec useActionState

Wszystkie formularze klienckie używają hooka useActionState z React 19:

```
const [state, action, pending] = useActionState(serverAction, undefined)

// state → poprzedni zwrot z akcji (błędy, komunikaty)
// action → handler przekazywany do form action={action}
// pending → boolean, true gdy akcja w toku (blokuje submit button)
```

10. Opisy widoków

/ — Strona główna

Typ: React Server Component

Layout: `(main)/layout.tsx` — nagłówek publiczny

Zawartość: Pusta strona — placeholder dla przyszłej landing page

Nagłówek: Nawigacja „For Athletes” / „For Brands” + przyciski SIGN IN / JOIN (lub DASHBOARD gdy zalogowany)

/login — Logowanie

Typ: Client Component (`'use client'`)

Elementy:

- Input `email` (type=email, autocomplete)
- Input `password` (type=password)
- Przycisk „Sign In” (disabled podczas pending)
- Link do rejestracji
- Komunikaty błędów inline przy każdym polu

Stan: `useActionState(signIn)`

/register — Rejestracja

Typ: Client Component

Elementy:

- Toggle buttons: **Athlete** / **Brand** — z wizualnym stanem active (`aria-pressed`)
- Input email + password
- Przycisk „Join” (disabled podczas pending)
- Link do logowania

Stan: `useState(accountType)` + `useActionState(signUp)`

Ukryte pole: `<input type="hidden" name="account-type" value={accountType} />`

/onboarding — Uzupełnienie profilu

Typ: Page — RSC, Form — Client Component

Elementy (Athlete): First name, Last name, DisciplinePicker

Elementy (Brand): Brand name, DisciplinePicker

DisciplinePicker: Toggle buttons dla Crossfit / Triathlon / Motorsports. Wybrany stan w `Set<Discipline>`. Emituje ukryte inputy.

Stan: `useActionState(completeOnboarding)`

10. Opisy widoków (cd.)

/dashboard — Panel główny

Typ: RSC

Guard: Wymaga profilu i ukończonego onboardingu

Zawartość: Pusta strona — placeholder. Nagłówek zawiera linki: My Campaigns, My Profile (placeholder) + SIGN OUT

/dashboard/campaigns — Lista kampanii

Typ: RSC

Dane: `getCampaigns()` — pobrane po stronie serwera

Elementy:

- Przycisk „New Campaign” (prawy górny róg)
- Stan pusty: komunikat + link do tworzenia
- Lista kart z: tytułem, opisem (2 linie), statusem jako badge, linkiem Preview

Kolejność: malejąca wg `created_at`

/dashboard/campaigns/new — Nowa kampania

Typ: Page — RSC, Form — Client Component

Elementy formularza:

- Title: input text, max 200 znaków
- Description: textarea 6 wierszy, max 5000 znaków
- DisciplinePicker (min 1 wymagana)
- Przycisk „Create Campaign” (pending: „Creating...”)

Validacja: Zod po stronie serwera — błędy inline przy polach

/dashboard/campaigns/:id — Szczegóły kampanii

Typ: RSC

Elementy:

- Link powrotu „← Back to Campaigns"
- Tytuł + badge statusu
- Opis (whitespace-pre-wrap, pełny tekst)
- Dyscypliny jako pill badges z labelami
- Daty Created / Last updated
- Przyciski: **Edit** (link indigo) + **Delete** (czerwony, z confirm dialog)

404: `notFound()` gdy kampania nie istnieje lub należy do innego usera

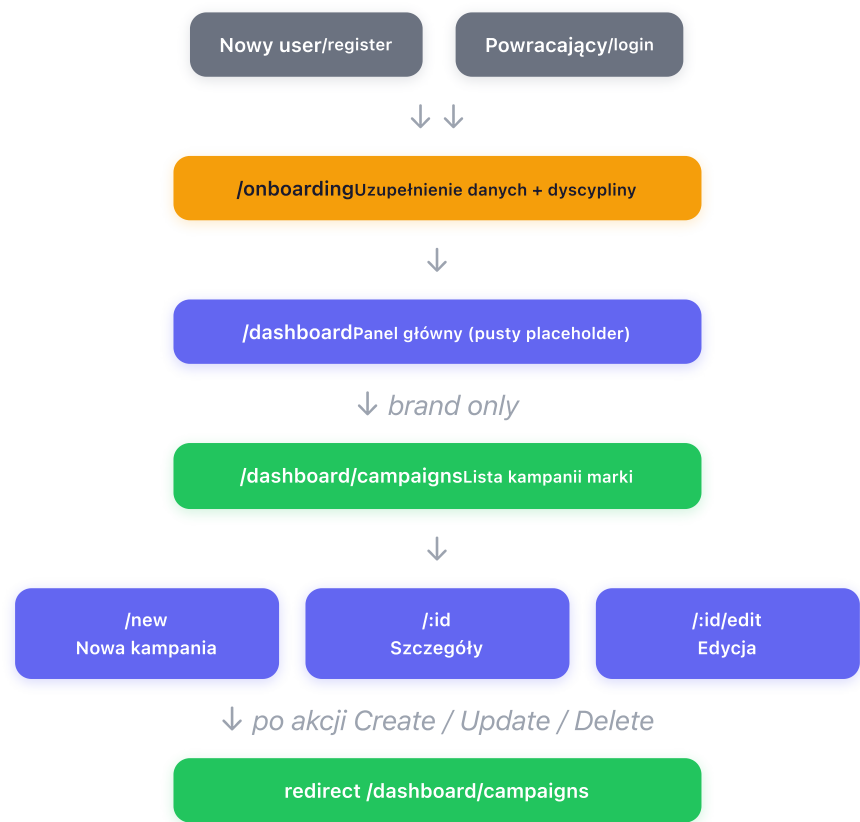
/dashboard/campaigns/:id/edit — Edycja kampanii

Typ: Page — RSC, Form — Client Component (ten sam CampaignForm)

Różnica vs /new: Pola prefillowane danymi z `campaign` prop. Akcja: `updateCampaign.bind(null, id)` . Przycisk: „Save Changes" / „Saving..."

DisciplinePicker: `defaultValue={campaign.disciplines ?? []}`

Podsumowanie — diagram całościowy



STAN IMPLEMENTACJI (MAJ 2026)

Zaimplementowane:

- Pełna autoryzacja (rejestracja, login, logout)
- Onboarding (athlete i brand)
- CRUD kampanii (brand)
- DisciplinePicker
- Guardy i przekierowania
- Walidacja Zod

Planowane / placeholder:

- Landing page (/)
- Dashboard główny (treść)
- My Profile (edycja profilu)
- Flow dla atletów (przeglądanie kampanii)
- System aplikowania atletów
- Weryfikacja email Supabase